



Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Sistemas e Industrial
Curso: Ingeniería de Software 1 (2016701)

Pruebas unitarias

Cristian Ochica

Para la realización de las pruebas se usó Pytest instalando esta herramienta en un entorno virtual del proyecto para realizar los tests directamente en la terminal, se usó el comando

```
pytest
```

Y para la medición de la cobertura el comando

```
pytest --cov=services --cov-report=term
```

Funcionalidades testeadas

1. Cálculo de resumen de venta

```
SaleService.calcular_resumen_venta()
```

Se realizó la prueba para que el sistema calcule correctamente los subtotales y el total general de una venta antes de la confirmación. Para esto se evaluaron los casos cuando se realiza la venta con un único videojuego, múltiples videojuegos y con un juego que no exista en la base de datos o no cuente con stock disponible. Se espera que el total corresponda a la suba de los subtotales de cada juego multiplicado por la cantidad y se debe generar una excepción cuando el juego no este disponible.

2. Registro de venta

```
SaleService.registrar_venta()
```

Se realizó la prueba para verificar que el sistema valide los principales procesos de la venta, es decir añadir uno o más juegos y que se carguen sus valores de cantidad disponible y precio unitario, la confirmación de venta y el posterior descuento del stock automáticamente. Se evaluaron los casos cuando se registra exitosamente una venta, cuando la venta no tiene detalles, es decir se hace una confirmación sin agregar nada al registro y cuando un juego no cuenta con el Stock suficiente. Se espera que la venta se registre correctamente cuando cuente con los requisitos necesarios y que genere una excepción cuando no exista la cantidad de productos disponibles.

3. Resumen diario de ventas

SaleService.obtener_resumen_diario()

Se realizó la prueba para verificar que el sistema consolide las ventas realizadas en una fecha determinada, para esto se evaluaron los casos cuando en un día se presentaron múltiples ventas y cuando no existió ninguna venta, se espera que la cantidad de ventas concuerde con la base de datos del sistema y el total recaudado en el día corresponda exactamente a la suma de los totales de las ventas.

Casos límite considerados

- Venta sin productos asociados
- Videojuegos inexistentes
- Stock insuficiente
- Múltiples productos dentro de una misma venta
- Resumen diario con múltiples ventas registradas
- Resumen diario sin ventas registradas

```
(venv) PS C:\Users\cris\Retrotrack - copia\project-backend> pytest --cov=services --cov-report=term
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.0, pluggy-1.6.0
rootdir: C:\Users\cris\Retrotrack - copia\project-backend
plugins: cov-7.1.0
collected 7 items

tests\test_sale_service.py ..... [100%]

===== warnings summary =====
database.py:13
  C:\Users\cris\Retrotrack - copia\project-backend\database.py:13: MovedIn20Warning: The ``declarative_base()`` function is now available
as sqlalchemy.orm.declarative_base(). (deprecated since: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
    Base = declarative_base()
ruff check .

All checks passed!
(venv) PS C:\Users\cris\Retrotrack - copia\project-backend> nings.html
* History restored ===== tests coverage =====
----- coverage: platform win32, python 3.13.5-final-0 -----
PS C:\Users\cris\Retrotrack - copia>
Name Stmts Miss Cover
-----
services\__init__.py 0 0 100%
services\auth_service.py 21 21 0%
services\genre_service.py 9 9 0%
services\juego_service.py 28 28 0%
services\platform_service.py 9 9 0%
services\sale_service.py 86 27 69%
services\usuario_service.py 47 47 0%
-----
TOTAL 200 141 30%
===== 7 passed, 1 warning in 0.93s =====
```

Como se observa los tests pasaron correctamente, las 3 funcionalidades corresponden al apartado `services\sale_service.py` y cuentan con una cobertura amplia respecto a esa carpeta y al total de las funcionalidades en la carpeta `service`.

*Los tests se encuentra en la ubicación `tests\test_sale_service.py` en el Backend de la aplicación